

# File Handling in C# with Examples

In this article, I am going to discuss **File Handling in C#** with Examples. At the end of this article, you will understand what is File Handling and why we need File Handling, and how to implement File Handling in C#.

## What is a File?

A file is a collection of data stored on a disk with a specific name, extension, and directory path. When you open File using C# for reading and writing purposes it becomes Stream.

## What is Stream?

A stream is a sequence of bytes travelling from a source to a destination over a communication path. There are two main streams: the Input Stream and the Output Stream. The input stream is used for reading data from the file (read operation) and the output stream is used for writing into the file (write operation).

1. **Input Stream:** This Stream is used to read data from a file, which is known as a read operation.
2. **Output Stream:** This Stream is used to write data into a file, which is known as a write operation.

So, in simple words, we can say that a stream is a sequence of bytes that is used for communication. When you open a file for reading or writing, it becomes a stream. There are two types of streams for a single file. One is the Input stream which is used for reading the file and the other one is the Output stream which is used for writing the file.

## Why do I need to Learn File Handling in C#?

As a C# Programmer, we need to save information on a disk. We will not get a database everywhere to save the information and our project may require saving information in a TEXT file, DOC file, XLS file, PDF file, or any other file type. So, we must know the concept of saving data in a file i.e. How to Handle Files in C#.

## File Handling in C#

Generally, we use the file to store data. The term File Handling in C# refers to the various operations that we can perform on a file such as creating a file, reading data from the file, writing data into the file, appending the file, etc.

Generally, we mostly perform three basic operations on a file: reading data from a file, writing data to a file and appending data to a file.

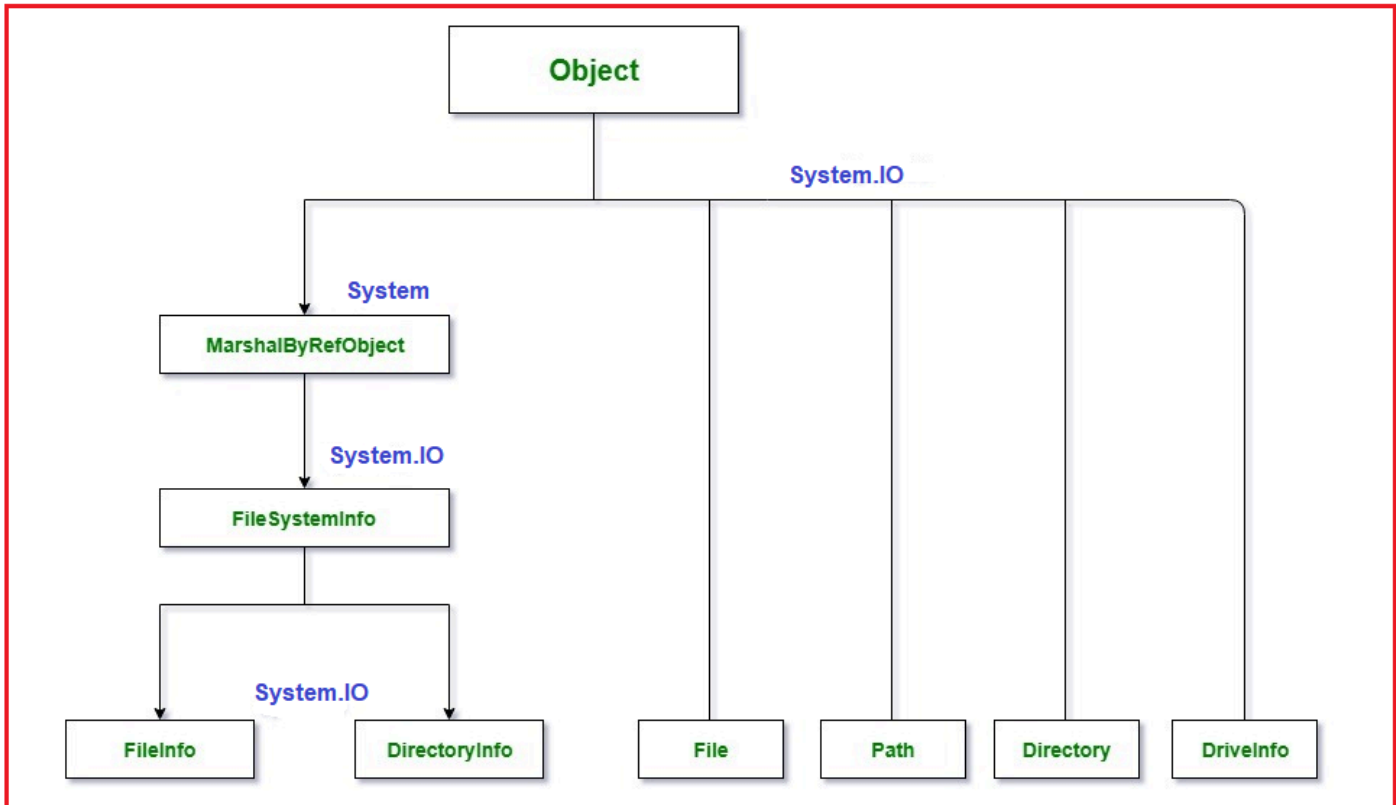
1. **Reading:** This operation is the basic read operation where the data is going to be read from a file.
2. **Writing:** This operation is the basic write operation where the data is going to be written to a file. By default, all existing contents are removed from the file, and new content is written in the file.
3. **Appending:** This operation is the same as the write operation where the data is going to be written to a file. The only difference between Write and Append is that the existing data in the file will not be overwritten. With Append, the new data is going to be added at the end of the file.

One more thing that you need to remember the file becomes a stream when we open the file either for writing or for reading.

## System.IO Namespace in C#

In C#, The System.IO namespace contains the required classes used to handle the input and output streams and provide information about file and directory structure. The parent class of file processing is Stream. Stream is an abstract class used as the parent of the classes that implement the necessary operations.

Please have a look at the below image which shows the file handling class hierarchy in C#.



**Note:** The FileInfo, DirectoryInfo, and DriveInfo classes have instance methods. File, Directory, and Path classes have static methods. The following table describes commonly used classes in the System.IO namespace.

| Class Name:    | Description                                                     |
|----------------|-----------------------------------------------------------------|
| FileStream:    | It is used to read from and write to any location within a file |
| BinaryReader:  | It is used to read primitive data types from a binary stream    |
| BinaryWriter:  | It is used to write primitive data types in binary format       |
| StreamReader:  | It is used to read characters from a byte Stream                |
| StreamWriter:  | It is used to write characters to a stream.                     |
| StringReader:  | It is used to read from a string buffer                         |
| StringWriter:  | It is used to write into a string buffer                        |
| DirectoryInfo: | It is used to perform operations on directories                 |
| FileInfo:      | It is used to perform operations on files                       |

In the next article, I am going to discuss [How to Implement File Handling using FileStream Class in C#](#) with Examples. Here, in this article, I try to explain **File Handling in C#** with examples. I hope you enjoy

this File Handling in C# article. I would like to have your feedback. Please post your feedback, question, or comments about this article.